Semana 8 — Desarrollo de Aplicaciones Móviles (DSY2204)

Formato de respuesta — Actividad Sumativa 3

Preparación de una aplicación móvil Android/Kotlin para su distribución

Nombre estudiante:	Victor Gonzalez Arenas	**Asignatura:**	DSY2204
Carrera:	Ing. en Desarrollo de Software	**Profesor:**	[Completar]
Fecha:	[Completar]	**Proyecto:**	VozApp

Datos del documento

Histórico de revisiones

Versión	Fecha	Descripción / cambio	Autor
1.0	[Fecha]	Sumativa 1: interfaz, navegación y arreglo de 5 usuarios.	Victor Gonzalez
2.0	[Fecha]	Sumativa 2: integración de funciones y colecciones de Kotlin.	Victor Gonzalez
3.0	[Fecha]	Sumativa 3: front+back con Firebase, KTX, views Escribir/Hablar/BuscarDispositivo, pruebas unitarias y APK firmado.	Victor Gonzalez

Información del proyecto

Organización	Duoc UC — Escuela de Informática y Telecomunicaciones
Sección	[Completar]
Proyecto (nombre)	VozApp
Fecha de inicio	[Completar]
Fecha de término	[Completar]
Patrocinador principal	[Completar]
Docente	[Completar]

1. Alcance del proyecto

Problema. Las personas con discapacidad sensorial auditiva enfrentan barreras de comunicación al interactuar con personas oyentes y con servicios digitales sin apoyos accesibles.

Solución propuesta. VozApp es una aplicación móvil de accesibilidad en Android/Kotlin que facilita la comunicación (escribir y hablar) y ubica dispositivos, integrando front end y back end.

Contexto de esta entrega (Sumativa 3). Se integra el back end con Firebase (Authentication y Firestore) para la persistencia y autenticación; se agregan las views HomeMenú, Escribir (texto a voz), Hablar (voz a texto) y BuscarDispositivo (geolocalización); se incorporan extensiones KTX; se desarrollan pruebas unitarias; y se prepara el APK firmado para su publicación en una plataforma de descarga.

2. Funciones esperadas del software (Mockups)

Prototipos de la autenticación y de las nuevas funciones principales.

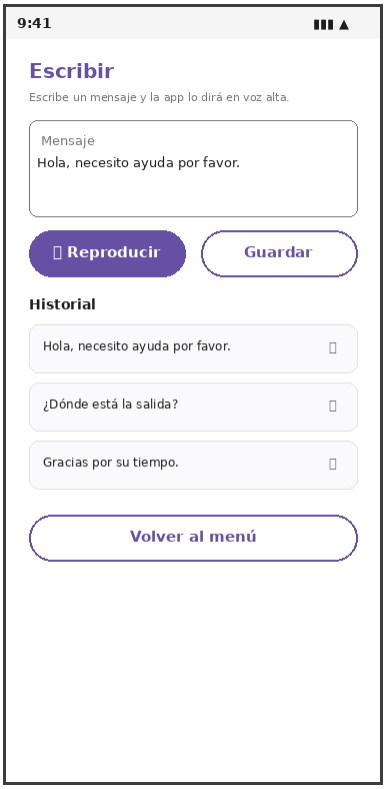
Login (autenticación)



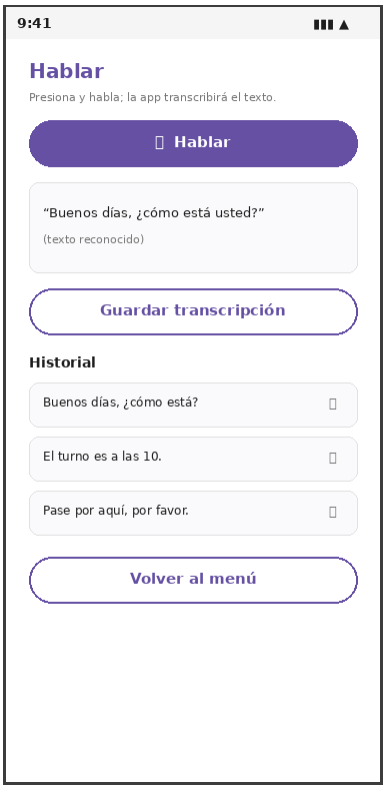
HomeMenú



Escribir (texto a voz)



Hablar (voz a texto)



BuscarDispositivo (geolocalización)



3. Rendimiento esperado — descripción por view

View	Descripción y rendimiento esperado
Login	Autentica con Firebase Auth mediante corrutinas; si el back end no está disponible, usa un respaldo local. Recomposición acotada a los campos.
Registro	Crea el usuario en Firebase Auth y guarda su perfil en Firestore (CRUD). Combo box alimentado por la enum TipoUsuario.
Recuperar contraseña	Envía correo de restablecimiento vía Firebase Auth. Pantalla ligera.
HomeMenú	Menú con tarjetas accesibles; navegación type-safe hacia las funciones. Jerarquía plana, alto rendimiento.
Escribir	Convierte texto a voz con TextToSpeech y guarda el mensaje en Firestore. La lista usa carga perezosa (LazyColumn).
Hablar	Transcribe voz a texto con SpeechRecognizer (permiso en runtime) y persiste la transcripción. Estado observable por StateFlow.
BuscarDispositivo	Obtiene la ubicación con Fused Location Provider de forma asíncrona y registra el dispositivo en Firestore (CRUD).

4. Integración de front end y back end

4.1. Extensiones KTX (Criterio 3)

Se integran extensiones KTX que simplifican el desarrollo: lifecycle-viewmodel-compose (viewModel() y viewModelScope), activity-ktx y fragment-ktx, core-ktx, y las extensiones de corrutinas para tareas de Play Services (await), que convierten las llamadas asíncronas de Firebase en código secuencial legible.

```
suspend fun iniciarSesion(correo, contrasena): Resultado<FirebaseUser> =
    auth.signInWithEmailAndPassword(correo, contrasena).await()  // KTX
corrutinas
```

4.2. Back end con Firebase (Criterio 5)

La persistencia y la autenticación se implementan con Firebase: Firebase Authentication para el registro/login/recuperación, y Cloud Firestore para los datos. La conexión se configura en app/google-services.json. Se implementan servicios y repositorios con operaciones CRUD completas:

Servicio / Repositorio	Responsabilidad (CRUD)
AuthService	Registro, inicio de sesión, recuperación y cierre de sesión (Firebase Auth).
UsuarioRemoteRepository	CRUD de perfiles de usuario en la colección 'usuarios'.
MensajeRepository	CRUD de mensajes de Escribir/Hablar en la colección 'mensajes'.
DispositivoRepository	CRUD de dispositivos localizados en la colección 'dispositivos'.

4.3. Servicios y funciones nativas

Escribir usa el motor TextToSpeech del sistema para leer el texto en voz alta; Hablar usa SpeechRecognizer para transcribir la voz; BuscarDispositivo usa el Fused Location Provider para la geolocalización. Los permisos de micrófono y ubicación se solicitan en tiempo de ejecución.

5. Pruebas unitarias

Se desarrollan pruebas unitarias en Kotlin con JUnit y Mockito, ejecutables con ./gradlew testDebugUnitTest:

Prueba	Qué verifica
UsuarioModelTest	Propiedades y funciones del modelo Usuario y la enum TipoUsuario (JUnit).
UsuarioRepositoryTest	Validación de credenciales, búsqueda, filtrado y registro sobre el arreglo (JUnit).
MensajesViewModelTest	Que el ViewModel crea/elimina y recarga la lista, simulando el repositorio CRUD con Mockito y corrutinas de prueba.

Además se incluye la base para pruebas con Robolectric y Espresso, y el proyecto queda preparado para Firebase Test Lab.

6. Restricciones del proyecto

- Adaptabilidad a múltiples dispositivos Android con conexión activa a internet.
- El back end (Firebase) requiere el archivo google-services.json del proyecto propio para operar en la nube.
- Las funciones de voz (TextToSpeech y SpeechRecognizer) y la geolocalización se prueban en un dispositivo o emulador real con los permisos concedidos.
- Formato del informe: Arial 12, interlineado 1,5, alineación a la izquierda.

7. Estructura de desglose de trabajo (EDT)

1. VozApp

1.1. Front end (views y navegación)
    1.1.1. Login, Registro, Recuperar contraseña
    1.1.2. HomeMenú, Escribir, Hablar, BuscarDispositivo
1.2. Back end (Firebase)
    1.2.1. Authentication (registro/login/recuperación)
    1.2.2. Firestore + CRUD (usuarios, mensajes, dispositivos)
1.3. Extensiones KTX y ViewModels
1.4. Funciones nativas (TextToSpeech, SpeechRecognizer, geolocalización)
1.5. Pruebas unitarias (JUnit + Mockito)
1.6. Generación, firma y publicación del APK
1.7. Control de versiones (Git) y documentación

Diccionario EDT:

Código	Paquete de trabajo	Descripción
1.1	Front end	Views en Compose y navegación type-safe entre ellas.
1.2	Back end Firebase	Autenticación y persistencia con CRUD en Firestore.
1.3	KTX y ViewModels	Extensiones KTX y gestión de estado con ViewModels y StateFlow.
1.4	Funciones nativas	Texto a voz, voz a texto y geolocalización.
1.5	Pruebas unitarias	JUnit y Mockito sobre modelo, repositorio y ViewModel.
1.6	APK	Generación, firma y publicación del APK.
1.7	Git y documentación	Repositorio y este informe técnico.

8. Seguimiento de los 5 riesgos técnicos

Seguimiento de los riesgos definidos en la Sumativa 1, con el resultado de los accionables de mitigación aplicados en esta entrega.

Riesgo	Probab.	Impacto	Estado	Acción de mitigación	Resultado del accionable
Fragmentación de Android	Media	Significativo	Controlado	minSdk 26 y pruebas en varias APIs	App estable en emulador y dispositivo; sin fallos por versión.
Pérdida de conexión	Alta	Alto	Mitigado	Operaciones asíncronas con manejo de errores (Resultado)	Las fallas de red se muestran como mensaje y no cierran la app.
Fugas de memoria	Media	Moderado	Controlado	Liberación de TTS/Speech en onDispose; estado en ViewModel	Sin fugas detectadas; recursos liberados al salir de la pantalla.
Latencia de autenticación/APIs	Media	Significativo	Mitigado	Indicadores de carga y corrutinas	Botón muestra 'Ingresando…'; UI no se bloquea durante la espera.
Vulnerabilidad de almacenamiento	Baja	Alto	Mitigado	Datos en Firestore con reglas de autenticación	El acceso a datos exige usuario autenticado (request.auth != null).

9. Generación, firma y publicación del APK

Generación (Criterio 6). Build → Build APK(s); el APK de depuración queda en app/build/outputs/apk/debug/.

Firma (Criterio 7). Build → Generate Signed Bundle / APK → APK → se crea un keystore (.jks) con contraseña y alias propios; el APK firmado queda en app/release/app-release.apk.

Publicación (Criterio 8). El APK firmado se publica en Firebase App Distribution (o en Releases de GitHub), que genera un enlace de descarga que simula la tienda.

Enlace de descarga del APK: _____ (completar tras publicar).

10. Definición de artefactos

Artefacto	Descripción
Código fuente (.zip desde Git)	Proyecto de Android Studio con front end, back end Firebase y pruebas.
APK firmado	Instalador de la app firmado con keystore propio.
Enlace de publicación	URL de descarga en la plataforma de distribución.
Repositorio Git	Historial de versiones del proyecto.
Informe técnico (este PDF)	Documentación del desarrollo, pruebas y seguimiento de riesgos.

11. Condiciones de aceptación para el cierre

- Las siete views se visualizan y navegan correctamente.
- Firebase Authentication valida el acceso y Firestore persiste usuarios, mensajes y dispositivos (CRUD).
- Escribir reproduce voz, Hablar transcribe y BuscarDispositivo obtiene la ubicación.
- Las pruebas unitarias pasan correctamente.
- El APK se genera, se firma y se publica con un enlace de descarga disponible.

Margen de tolerancia: se aceptan defectos menores de estilo visual que no afecten la funcionalidad ni la navegación.